SQL Basics — Retail Mart Management: Study Guide

A concept-by-concept walkthrough of the 16 exercises, using the actual `product`, `customer`, and `sales` tables. Each section explains **what the command does**, **why it's used**, and shows the **real result** from this dataset so you can check your own queries against it.

1–2. Creating and selecting a database

```
CREATE DATABASE sql_basics;
USE sql_basics;
```

- `CREATE DATABASE` makes a new, empty database on the server.
- `USE` tells the server "run all my following commands against this database" — you need this before creating tables, or every table name has to be fully qualified (`sql_basics.product`).
- **Note:** SQL identifiers can't contain spaces, so "SQL basics" becomes `sql_basics`.

3. Creating tables (DDL — Data Definition Language)

```
CREATE TABLE product (
    p_code    INT PRIMARY KEY,
    p_name    VARCHAR(50),
    price     DECIMAL(10,2),
    stock     INT,
    category  VARCHAR(50)
);
```

- `PRIMARY KEY` marks the column that uniquely identifies each row — no duplicates, no NULLs allowed.
- `DECIMAL(10,2)` is the right type for money: exact, no floating-point rounding errors (unlike `FLOAT`).
- Same pattern for `customer` and `sales`. In `sales`, `p_code` and `c_id` are **foreign-key-style** columns — they reference rows in `product` and `customer`, which is how the tables relate to each other later on (see the JOIN in section 16).

4. Inserting data (DML — Data Manipulation Language)

```
INSERT INTO product (p_code, p_name, price, stock, category)
VALUES (1, 'tulip', 198, 5, 'perfume');
```

- One `INSERT` can carry multiple rows separated by commas — much faster than one statement per row.
- Column order in `VALUES` must match the column list right after the table name.

5. Adding columns — `ALTER TABLE ... ADD COLUMN`

```sql
ALTER TABLE sales
    ADD COLUMN S_no INT,
    ADD COLUMN categories VARCHAR(50);
```

- `ALTER TABLE` changes a table's structure *after* it's been created, without losing existing data.
- New columns start out `NULL` for every existing row until you populate them.

6. Changing a column's type — `MODIFY COLUMN`

```sql
ALTER TABLE product
    MODIFY COLUMN stock VARCHAR(50);
```

- `MODIFY COLUMN` changes type/size in place. Here `stock` goes from `INT` to `VARCHAR` — useful if you later need to store non-numeric values like `"out of stock"` or `"12 (backordered)"`.
- Watch out: shrinking a type or changing types can silently truncate or reject data that doesn't fit the new type.

7. Renaming a table

```sql
RENAME TABLE customer TO customer_details;
```

- Renames the table; all data, indexes, and constraints carry over.
- After this, every later query must use the new name — that's why every query from here on in the exercise refers to `customer_details`, not `customer`.

8. Dropping columns

```sql
ALTER TABLE sales
    DROP COLUMN S_no,
    DROP COLUMN categories;
```

- Permanently removes the column **and its data**. There's no undo — this is why `DROP` commands are the ones to be most careful with.

9. Selecting specific columns + aliasing

```sql
SELECT
    order_no   AS order_id,
    c_id       AS customer_id,
    order_date,
    price,
    qty
FROM sales;
```

Result (first 3 of 10 rows):

order_id	customer_id	order_date	price	qty
HM06	9212	2016-07-24	30	3
HM09	3921	2016-10-19	600	10
HM10	9875	2016-10-30	500	10

- `AS` renames a column *only in the output* — it doesn't touch the actual table. Handy when the question asks for "order id" but your column is really called `order_no`.

10. Filtering with `WHERE`

```sql
SELECT * FROM product WHERE category = 'Stationary';
```

➔ Returns 6 rows: Pen, pencil, sharpener, sketch pen, tape, paint.

- `WHERE` filters *rows*, evaluated before the results are returned. String comparisons are case-sensitive in some engines — worth checking your specific SQL flavor.

11. `DISTINCT`

```sql
SELECT DISTINCT category FROM product;
```

➔ 10 unique categories (perfume, icecream, Stationary, snacks, dip, spread, hair product, fruits, vegetable, kitchen utensil) instead of 26 repeated rows.

- `DISTINCT` removes duplicate rows *from the result set only* — the underlying table is untouched.

12. Combining conditions with `AND`

```sql
SELECT * FROM sales WHERE qty > 2 AND price < 500;
```

➔ 3 rows: pencil (qty 3, ₹30), kiwi (qty 3, ₹420), mayanoise (qty 4, ₹360).

- `AND` requires *both* conditions true. Using `OR` instead would return a much larger, looser set — a common source of "why did my query return too many rows" bugs.

13. Pattern matching with `LIKE`

```sql
SELECT c_name FROM customer_details WHERE c_name LIKE '%a';
```

➔ Nisha, Nila, Rithika, Jessica.

- `%` is a wildcard for "any sequence of characters (including none)." `'%a'` = ends with "a". `'a%'` would mean *starts* with "a", and `'%a%'` would mean "contains a" anywhere.

14. Sorting with `ORDER BY`

```sql
SELECT * FROM product ORDER BY price DESC;
```

➔ Highest first: park avenue (901) ➔ axe (210) ➔ wattagirl (201) ➔ conditioner (200) ➔ tulip (198) ➔ ...

- `DESC` = descending (high to low). Leave it off, or use `ASC`, for ascending (low to high), which is the default.

15. `GROUP BY` + `HAVING` + subquery

```sql
SELECT p_code, category
FROM product
WHERE category IN (
    SELECT category
    FROM product
    GROUP BY category
    HAVING COUNT(*) >= 2
)
ORDER BY category;
```

Why not just `GROUP BY category HAVING COUNT(*) >= 2` directly? Because grouping collapses all rows in a category into *one* summary row — you'd get `category` and a count, but not the individual `p_code` values inside each group. The subquery first finds *which categories* qualify (appear 2+ times), then the outer query goes back to the un-grouped table to pull every product code in those categories.

➔ Returns Stationary (6 codes), fruits (3), hair product (3), perfume (4), snacks (3), vegetable (3) — every category with 2 or more products, one row per product.

- `HAVING` is like `WHERE`, but for *groups* instead of individual rows — you can't use `WHERE COUNT(*) >= 2` because `WHERE` runs before grouping happens.

16. Combining two result sets: `UNION ALL` vs. `JOIN`

`UNION ALL` stacks two queries' results on top of each other — they must have the same number of columns:

```
SELECT order_no, c_name FROM sales
UNION ALL
SELECT NULL, c_name FROM customer_details;
```

➔ 23 rows total (10 from sales + 13 from customer_details), duplicates kept. (Plain `UNION`, without `ALL`, would remove exact-duplicate rows across the two sets.)

`JOIN` instead *matches* rows between tables using a shared key — which is almost always what you actually want when relating an order to "its" customer:

```
SELECT s.order_no, c.c_name
FROM sales s
JOIN customer_details c ON s.c_id = c.c_id;
```

➔ 10 rows, each order paired with the correct customer name via `c_id`.

- **The key distinction to remember for exams:** `UNION` / `UNION ALL` stack unrelated queries vertically (same shape, different source); `JOIN` combines related tables horizontally (matching on a key). If a question says "combine two tables" but really means "for each order, show its customer," that's a JOIN, not a UNION — a common exam trap.

Quick reference: keyword cheat sheet

Keyword	Purpose
`CREATE DATABASE / USE`	Make and select a database
`CREATE TABLE`	Define a new table's structure
`INSERT INTO`	Add rows
`ALTER TABLE ... ADD COLUMN`	Add a column
`ALTER TABLE ... MODIFY COLUMN`	Change a column's type
`RENAME TABLE`	Rename a table
`ALTER TABLE ... DROP COLUMN`	Delete a column (and its data)
`SELECT ... AS`	Choose columns, rename in output
`WHERE`	Filter rows
`DISTINCT`	Remove duplicate rows from output
`AND / OR`	Combine conditions
`LIKE '%x%'`	Pattern match text
`ORDER BY ... ASC/DESC`	Sort results
`GROUP BY + HAVING`	Summarize rows into groups, then filter groups
`UNION ALL`	Stack two same-shaped result sets, keep duplicates
`JOIN ... ON`	Match rows between tables on a key

Companion files: `retail_mart_management.sql` *(the runnable script) and* `retail_mart_db` *(a ready-to-query SQLite database with this data already loaded).*